
FOUNDATIONS OF MEMORY HIERARCHY DESIGN

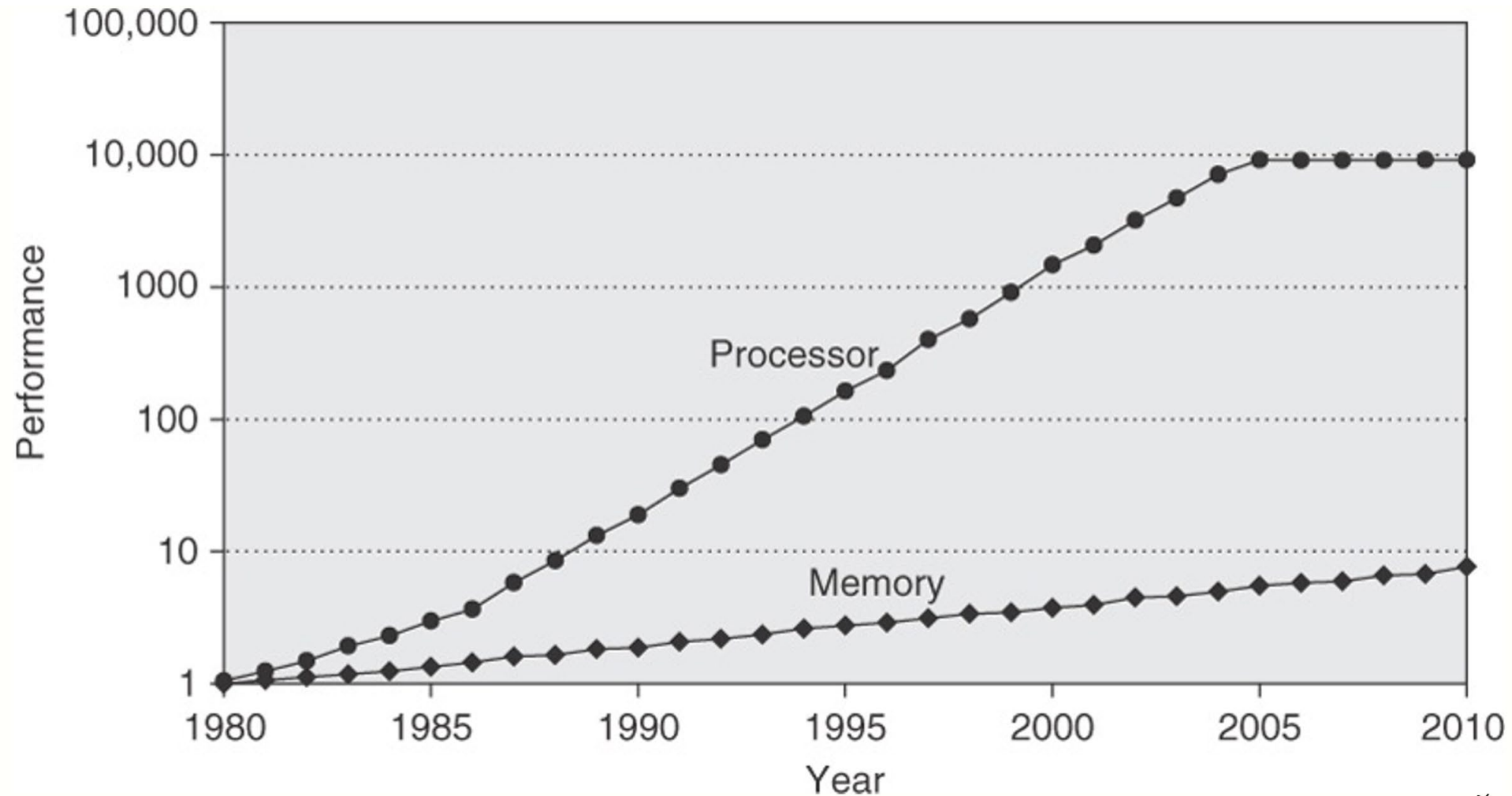
Why memory matters today

The Real Bottleneck:

- CPU frequency scaling slowed
- Core counts increased
- Memory latency improved very slowly
- Data-intensive workloads dominate

Performance today is often limited by memory behavior, not raw compute.

Why memory matters today



Copyright © 2011, Elsevier Inc. All rights Reserved.

Why memory dominates memory systems

The time to execute a program is given by

$$CPU_{time} = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$$

CPI can be expanded in

$$CPI = CPI_{base} + CPI_{memory} + CPI_{branch} + \dots$$

Why memory dominates memory systems

Let us focus on the CPI

$$CPI = CPI_{base} + CPI_{memory} + CPI_{branch} + \dots$$

In early processor the CPI_{base} was dominant, but today CPI_{memory} dominates

Processor performance improved dramatically over decade, but DRAM latency did not scale at the same rate

Why memory dominates memory systems

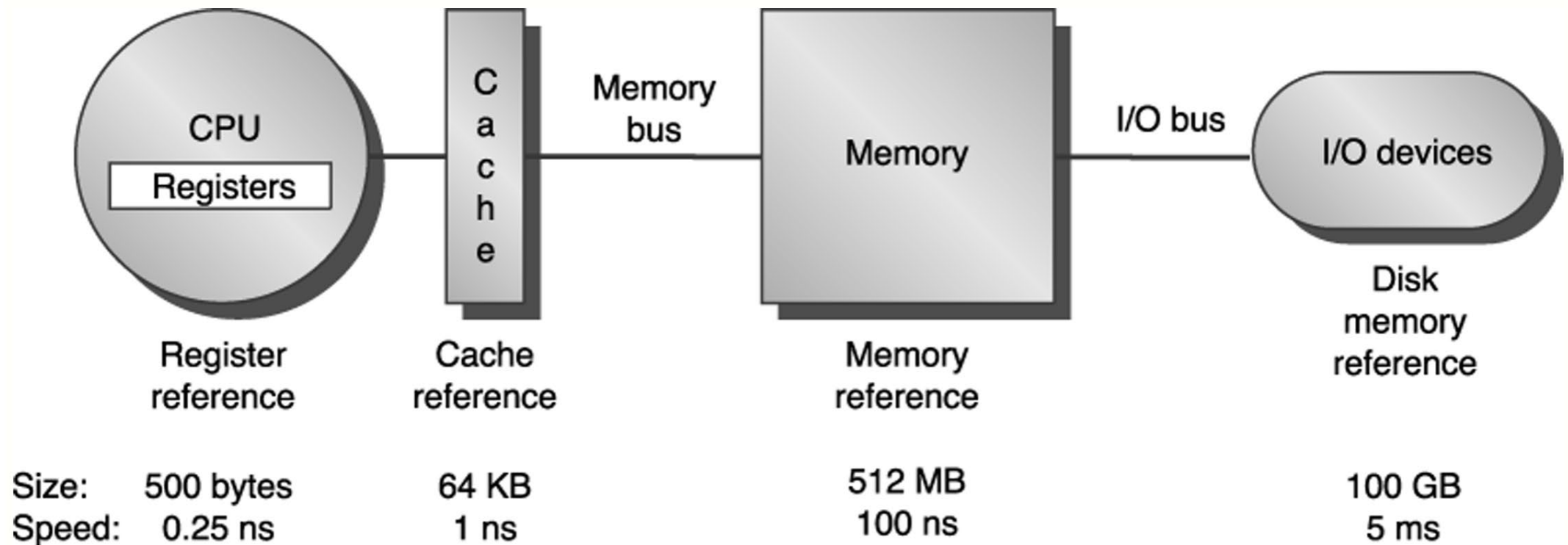
This phenom is known as Memory Wall



We cannot simply build faster CPUs

We must design smarter memory systems

Why memory dominates memory systems



Latency vs Bandwidth

Latency -> time to complete a single memory access

Bandwidth -> data transferred per unit time

Examples:

GPUs have enormous bandwidth, yet the individual memory access may still have significant latency

SSDs provide high throughput, but random access latency is orders of magnitude higher than DRAM

Locality Refresher

The memory hierarchy exists because of one fundamental property: locality

Temporal locality (if you accessed data recently, you will likely to access it again soon)

Spatial locality (if you access address X, you will likely access nearby addresses)

Locality Refresher

Locality exists because programs are structured:

- Loops
- Contiguous data structures
- Stack disciplines
- Cache-friendly compilation

If programs had no locality, caches would be useless!

Working Set Concepts

The working set is the set of memory locations actively used during a time window

Working set fits in cache -> low miss rate

Working set exceeds cache -> thrashing

Cache size decisions depend on

- Target workload
- Expected working set distribution
- Cost and energy constraints

Memory Hierarchy Structure

Single-level memory design is infeasible

Fast memory:

- Expensive
- Large area
- High leakage

Large memory:

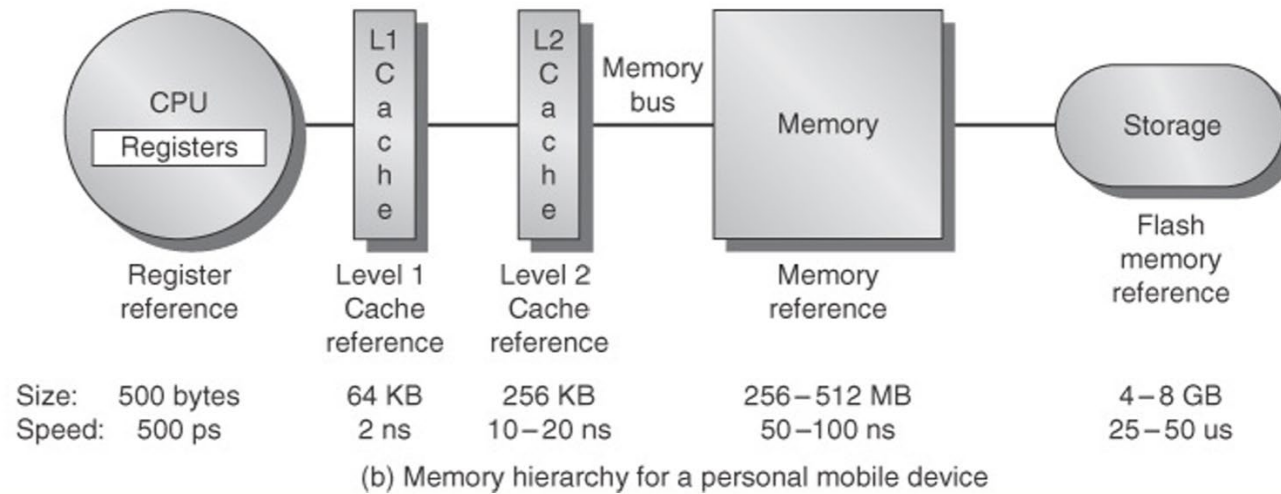
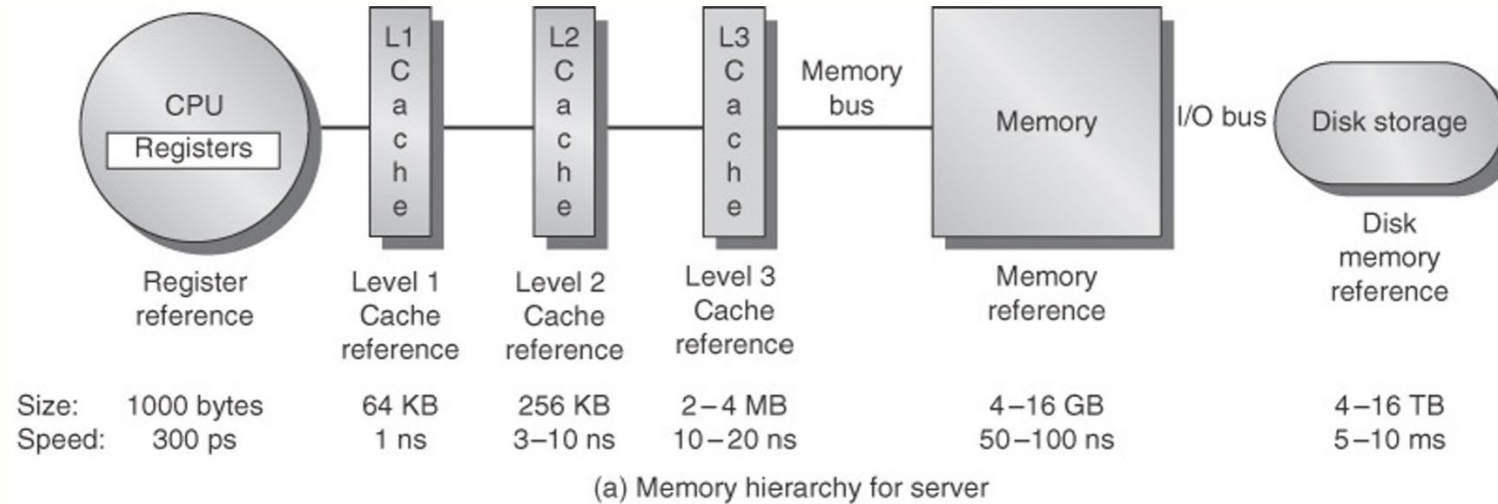
- Slow
- High latency

Memory Hierarchy Structure

Typical structure

- Registers (~1 cycle)
- L1 cache (~1-4 cycles)
- L2 cache (~10 cycles)
- LLC – Last Level Cache (~30-50 cycles)
- DRAM (~80-150+ ns)
- Storage (microseconds to milliseconds)

Memory Hierarchy Structure



Average Memory Access Time

A first fundamental model can be introduced

$$AMAT = HitTime + MissRate \times MissPenalty$$

Examples

Hit time = 1 cycle

Miss rate = 5%

Miss penalty = 50 cycles

$$AMAT = 1 + 0.05 \times 50 = 3.5$$

Miss penalty = 200 cycles

$$AMAT = 1 + 0.05 \times 200 = 11$$

Average Memory Access Time

As hierarchies deepen miss penalty dominates

What to do?

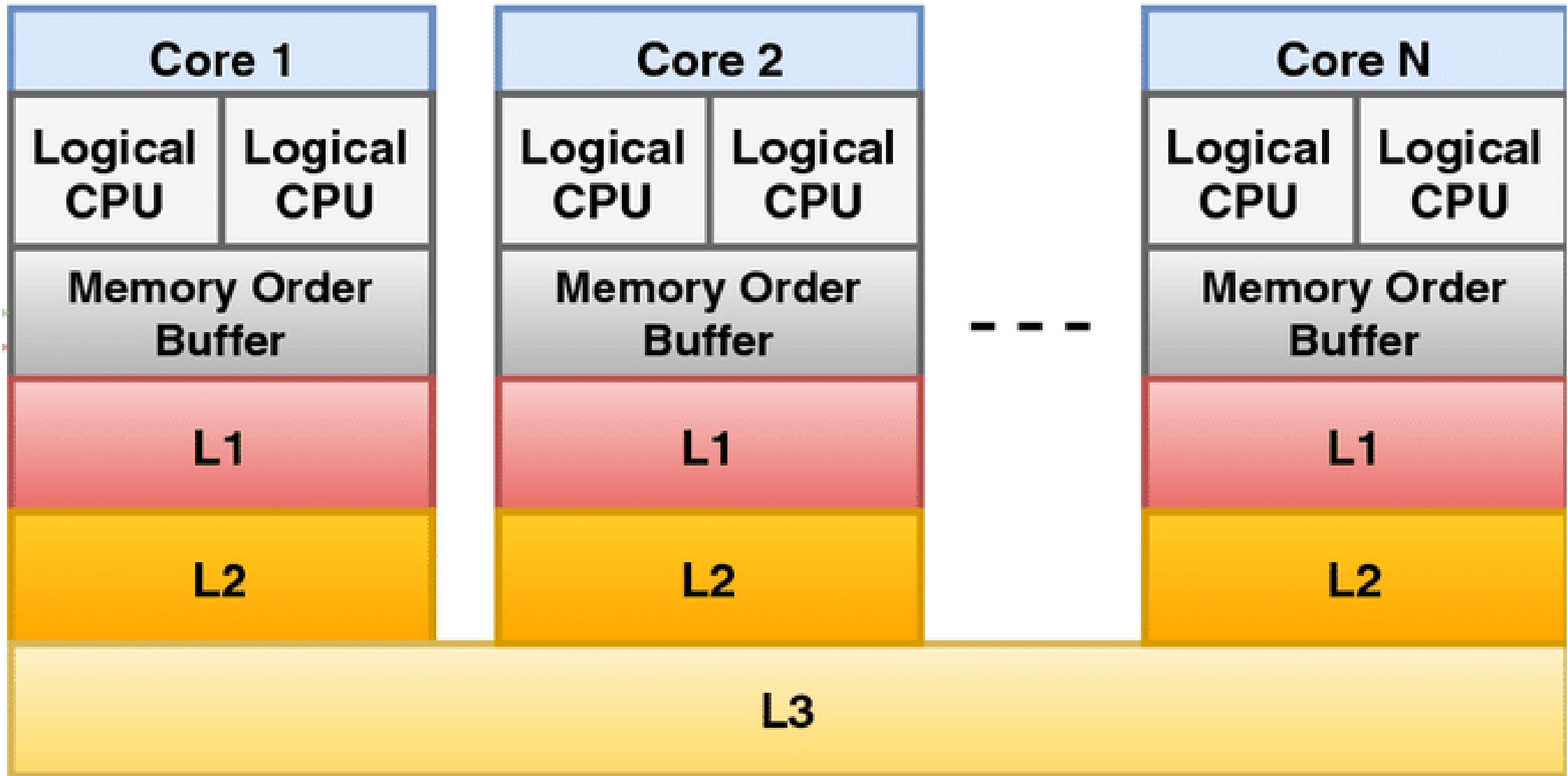
- Reducing miss penalty
- Overlapping misses
- Exploiting memory-level parallelism

Modern Multicore System View

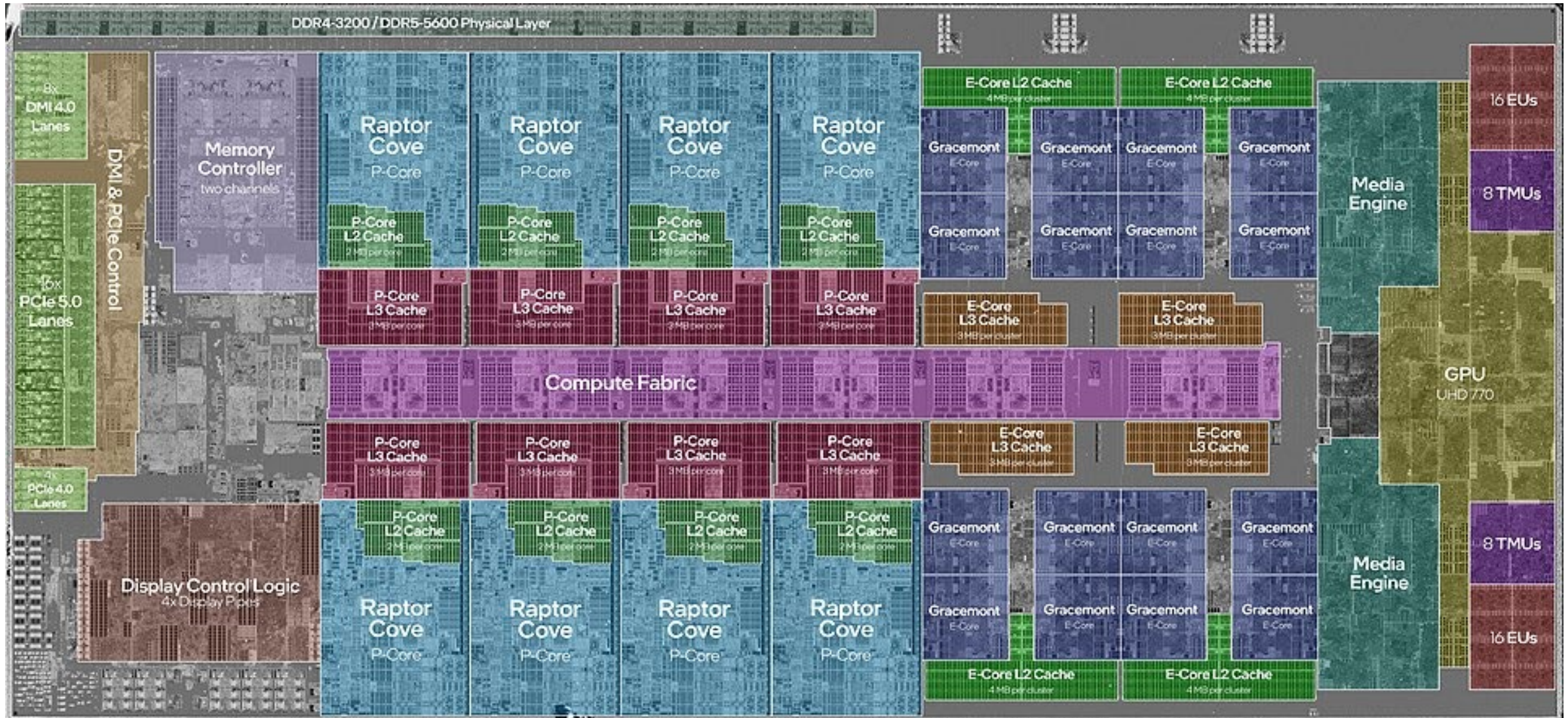
In a modern multicore processor you can observe

- multiple cores
- Private L1 and L2 caches
- Shared L3 cache
- Integrated memory controller
- On-chip interconnect

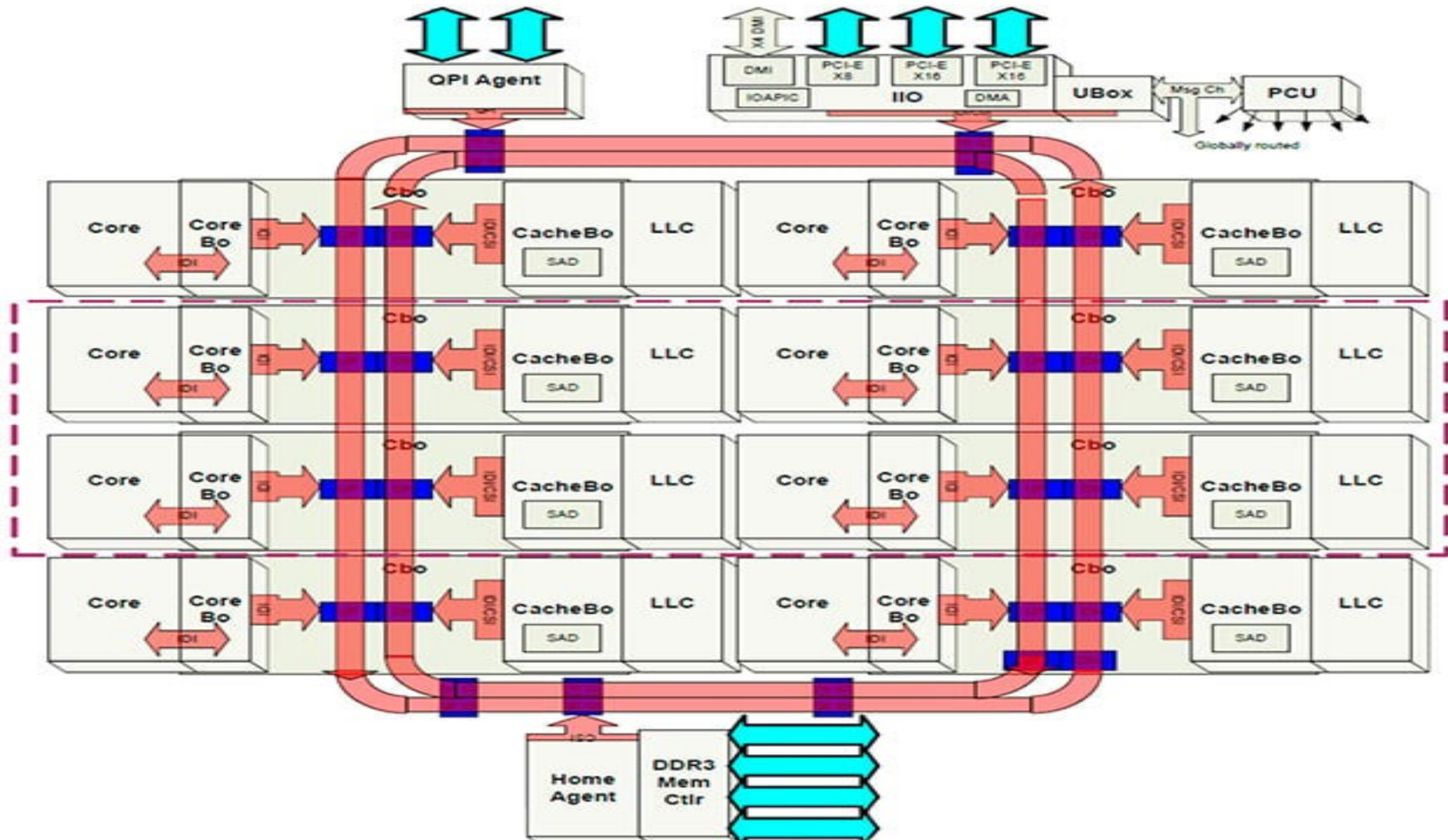
Modern Multicore System View



Modern Multicore System View – i9 13900K



Modern Multicore System View – Intel Xeon



DRAM as a Structured System

DRAM is not a flat array!

It contains:

- Banks
- Rows
- Row buffer

Row activation is expensive and access pattern matters!

Closing

Take home messages:

- Memory dominates modern performance
- Locality enables hierarchy
- Working set matter
- AMAT provides modelling framework
- Memory design is a multi-objective optimization

Next episode: we begin analyzing real cache organization

PROBLEM SET 1

MEMORY FUNDAMENTALS AND AMAT

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

1. Compute AMAT.
2. If miss rate is reduced to 3% compute new AMAT.
3. If miss penalty is reduced by 60 cycles instead, compute AMAT.
4. Which improvements provides larger benefit? Explain.

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

1. Compute AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.04 \times 100 = 5 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

2. If miss rate is reduced to 3% compute new AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.03 \times 100 = 4 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

3. If miss penalty is reduced by 60 cycles instead, compute AMAT.

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.04 \times 60 = 3.4 \end{aligned}$$

Problem 1- AMAT Sensitivity Analysis

A processor has L1 hit time of 1 cycle, L1 miss rate of 4% and a miss penalty to memory of 100 cycles.

4. Which improvements provides larger benefit? Explain.

Reducing miss rate -> improvement of 1 cycle

Reducing miss penalty -> improvement of 1.6 cycles

When miss penalty is large even a moderate miss rate causes many stalls cycles (deep hierarchy, DRAM access).

Cutting penalty directly reduces the cost per miss which often dominates total memory time.

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

1. Compute memory stall cycles per instruction.
2. Compute total CPI.
3. If miss penalty doubles, what happens to total CPI?

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

1. Compute memory stall cycles per instruction.

First compute miss frequency per instruction:

$$P(\text{miss per instruction}) = 0.30 \times 0.10 = 0.03$$

(3% of the instructions incur a miss)

$$\text{stall cycles per instruction} = 0.03 \times 50 = 1.5$$

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

2. Compute total CPI.

$$CPI = CPI_{base} + CPI_{memory} = 1 + 1.5 = 2.5$$

Problem 2- CPI Impact of Memory Stalls

Consider a base CPI of 1.0 and knowing that 30% of instructions are loads and 10% of these loads miss in the L1 cache with a miss penalty of 50 cycle.

3. If miss penalty doubles, what happens to total CPI?

$$CPI_{memory} = 0.03 \times 100 = 3$$

$$CPI = CPI_{base} + CPI_{memory} = 1 + 3 = 4$$